CS 3510: Design & Analysis of Algorithms 11/4/2024

Problem Set 8: NP-completeness II

Prof. Abrahim Ladha Due: 11/11/2024 11:59pm

- **Please TYPE your solutions using Latex or any other software. Handwritten solutions *won't* be accepted.**

Steps to write a reduction

To show that a problem is NP-complete, you need to show that the problem is both in NP and NP-hard. In a polynomial-time reduction, we have a *Problem B*, and we want to show that it is NP-complete. These are the steps:

- Demonstrate that *problem B* is in the class NP. This is a description of a procedure that verifies a candidate's solution. It needs to address the runtime. You should give a polytime verifier which takes as input a candidate problem, and a witness, and outputs true or false if the witness is a solution.
- Demonstrate that *problem B* is at least as hard as a problem previously proved to be NP-complete. Choose some NP-complete A and prove $A \leq_p B$. You need to show that *problem B* is NP-hard. This is done via polytime reduction from a known NP-complete *problem A* to the unknown *problem B* ($A \to B$) as follows:
  1. Choose A. You will have many NP-complete problems to pick from later on, and it is best to pick a similar one.
  2. Give your reduction f and argue that it runs in polynomial time.
  3. Prove that $x \in A \implies f(x) \in B$
  4. Prove that $x \notin A \implies f(x) \notin B$
  5. This is sufficient to show $x \in A \iff f(x) \in B$. Since A was NP-complete, and $A \leq_p B$, we can conclude that B is NP-hard. Note: You don't have to provide a formal proof. You can briefly explain in words both implications and why they hold.

The second bullet point above is prove that *problem B* is NP-hard which combined with the first bullet point yields the NP-complete proof.

Problem 1
(25 points)

For an undirected graph $G = (V, E)$, **a strongly independent set** is a set S of vertices such that $S \subseteq V$ and for any two vertices $u, v \in S$ there is no path of length ≤ 2 between u and v. For any path, its length is equal to the number of edges in that path. Consider the following `Strongly-Independent-Set` problem:

Input: An undirected graph $G = (V, E)$ and an integer $k > 0$.
Output: Whether a strongly independent set of size k in G exists.

Prove that this problem is NP-Complete.

Solution: To prove that the Strongly-Independent-Set problem is NP-Complete, we follow the following logic:

We recognize we can verify whether S is a strongly independent set of size k in polynomial time.

We begin by checking if $|S| = k$. Then, for every pair of vertices $u, v \in S$ $(u \neq v)$, we verify there is no edge $(u, v) \in E$ and for every vertex $w \in V$, we check if $(u, w) \in E$ and $(w, v) \in E$.

We observe the number of vertex pairs in S is $\binom{k}{2} = O(k^2)$. For each pair, checking for direct edges takes $O(1)$ time. Additionally, checking for paths of length 2 requires iterating over all $w \in V$, which is $O(|V|)$ per pair. Therefore, the time complexity is $O(k^2|V|)$, where $k \leq |V|$. Hence, the problem is in NP.

We reduce the *Independent Set* problem, which is NP-Complete, to the Strongly-Independent-Set problem.

Given an instance (G, k) of the Independent Set problem, we construct an instance (G', k) of the Strongly-Independent-Set problem by subdividing each edge in G. In Particular, for each edge $(u, v) \in E(G)$, we introduce a new vertex w_{uv} and replace (u, v) with two edges (u, w_{uv}) and (w_{uv}, v).

In the resulting graph G', any independent set of size k in G corresponds to a strongly independent set of size k in G', as there are no direct edges or paths of length 2 between the vertices in the set. Similarly, any strongly independent set in G' of size k corresponds to an independent set in G of size k.

Since the reduction can be performed in poly-time, and the Independent Set problem is NP-Complete, the Strongly-Independent-Set problem is NP-hard.

Strongly-Independent-Set problem is in NP and NP-hard, it is NP-Complete.

Problem 2
(25 points)

Consider the `Busy-Schedule` problem:

Input: A list of n classes with multiple meeting times (each meeting time represented by an interval $[a, b]$ where $a < b$ and a, b positive integers), a start time s, an end time e, and an integer k.
Output: Whether there exists a k-sized selection S of the n classes such that at any time between s and e at least one class in S is meeting.

Show that the `Busy-Schedule` problem is NP-complete.

Solution: To prove that the Busy-Schedule problem is NP-Complete, we follow the following logic:

We recognize we can verify whether a selection S of k classes satisfies the busy schedule condition in polynomial time.

We begin by checking if $|S| = k$. Then, for each time point t between s and e, we verify that there exists at least one class in S that is meeting at time t. This involves checking, for each class in S, whether t lies within any of its meeting intervals $[a, b]$.

We observe that there are at most $e - s + 1$ time points to check. For each time point t, verifying the existence of a meeting class in S takes $O(k)$ time. Therefore, the time complexity is $O(k \cdot (e - s + 1))$, which is polynomial since $k \leq n$ and $e - s + 1$ is bounded by the input size. Therefore, we find, the problem is in NP.

We reduce the Vertex Cover problem, which we know from lecture to be NP-Complete, to the Busy-Schedule problem.

Given an instance (G, k) of the Vertex Cover problem, we construct an instance (C, s, e, k') of the Busy-Schedule problem as follows:

For each edge $e_i = (u_i, v_i) \in E$, assign a distinct time slot t_i. Let $T = \{t_1, t_2, \ldots, t_m\}$, where $m = |E|$. Then, we set the overall time interval as $s = 1$ and $e = m$. Next, for each vertex $v \in V$, create a class C_v. Finally, for each class C_v, its meeting times are the time slots corresponding to the edges incident to v. Specifically, for each edge e_i incident to v, include the interval $[t_i, t_i]$. Even more finally, we set $k' = k$.

We find assigning time slots to edges takes $O(|E|)$ time. Similarly, creating classes and their meeting times involves iterating through each vertex and its incident edges, which takes $O(|V| + |E|)$ time. Operations regarding the start and end times occur in $O(1)$. Therefore, we find the reduction runs in poly-time.

Now, we will prove that $x \in A \implies f(x) \in B$.

If there exists a vertex cover $C \subseteq V$ of size at most k in G, then we select the corresponding

classes $S = \{C_v \mid v \in C\}$ in the Busy-Schedule problem. For each edge $e_i = (u_i, v_i) \in E$, at least one of u_i or v_i is in C, since C is a vertex cover.Therefore, the time slot t_i corresponding to e_i is covered by at least one class in S, because the class corresponding to the vertex in C that is incident to e_i meets at time t_i. Since every edge in E is covered, every time slot t_i for $i = 1, \ldots, m$ is covered by at least one class in S. Therefore, at any time between $s = 1$ and $e = m$, at least one class in S is meeting. Therefore, $f(G, k) = (C, s, e, k)$ is a "yes" instance of the Busy-Schedule problem.

Now, we will prove that $x \notin A \implies f(x) \notin B$.

If there does not exist a vertex cover of size k in G, then any selection of k classes in the Busy-Schedule problem corresponds to selecting k vertices in G. Since there is no vertex cover of size k, there exists at least one edge $e_i = (u_i, v_i) \in E$ such that neither u_i nor v_i is included in any selection of k vertices. This means that the time slot t_i corresponding to edge e_i is not covered by any class in the selection S, since neither C_{u_i} nor C_{v_i} is in S. Therefore, at time t_i, no class in S is meeting. Thus, S does not satisfy the busy schedule condition. Therefore, we find $f(G, k) = (C, s, e, k)$ is a "no" instance of the Busy-Schedule problem.

Since $x \in A$ if and only if $f(x) \in B$, and the reduction f is computable in polynomial time, the Busy-Schedule problem is NP-hard.

Therefore, since we have shown that the Busy-Schedule problem is in NP and since we have reduced the NP-Complete Vertex Cover problem to it, we find the Busy-Schedule problem is NP-Complete.

Problem 3
(25 points)

Say we are given n ingredients numbered 1 to n. There exists an $n \times n$ matrix M that represents the *deliciousness* between any two ingredients. Between any two ingredients a, b, the *deliciousness* value $M[a, b]$ ranges from 0.0 to 1.0 (0.0 meaning that a and b do not go well with each other and 1.0 meaning that a and b are a perfect match). M is symmetric ($M[a, b] = M[b, a]$) and has zeros along its diagonal ($M[a, a] = 0$). Here is an example matrix with 3 ingredients:

	1	2	3
1	0.0	0.4	0.2
2	0.4	0.0	0.1
3	0.2	0.1	0.0

We want to make a selection of k of these ingredients S such that the sum of the *deliciousness* between all pairs of ingredients in S is maximized. Consider the `Best-Dish` problem:

Input: The number of ingredients n, the *deliciousness* matrix M, a threshold m, and an integer k
Output: Whether there exists a k-sized subset of the n ingredients S such that the total *deliciousness* of S is $\geq m$.

Show that the `Best-Dish` problem is NP-complete.

Solution: To prove that the Best-Dish problem is NP-Complete, we follow the following logic:

We recognize we can verify whether a subset S of k ingredients has total deliciousness at least m in polynomial time.

We begin by checking if $|S| = k$. Then, we compute the total deliciousness of S by summing $M[a, b]$ for all pairs $a, b \in S$ with $a < b$.

We observe that the number of pairs is $\binom{k}{2} = O(k^2)$. Each $M[a, b]$ lookup takes $O(1)$ time. Therefore, the time complexity is $O(k^2)$, which is polynomial since $k \leq n$. Hence, the problem is in NP.

We reduce the Clique problem, which is NP-Complete, to the Best-Dish problem.

Given an instance (G, k) of the Clique problem, we construct an instance (n, M, m, k') of the Best-Dish problem as follows:

We first let $n = |V|$.For each vertex $v \in V$, create an ingredient. Construct the deliciousness matrix M where for all $a, b \in V$:if $(a, b) \in E$, then $M[a, b] = 1$; if $(a, b) \notin E$ or $a = b$, then $M[a, b] = 0$. Next, we set the threshold $m = \dfrac{k(k-1)}{2}$. Finally, we set $k' = k$.

We observe constructing M involves iterating over all pairs (a, b) in $V \times V$, which takes $O(n^2)$ time. Therefore, we find the reduction runs in poly-time.

Now, we will prove that $x \in A \implies f(x) \in B$.

If G has a clique C of size k, then for every pair $a, b \in C$, $a \neq b$, we have $M[a, b] = 1$. The total deliciousness of C is found by the following:

$$\text{Total Deliciousness} = \sum_{\substack{a,b \in C \\ a<b}} M[a, b] = \binom{k}{2} = \frac{k(k-1)}{2} = m.$$

Therefore, we find $S = C$ is a subset of size k with total deliciousness $\geq m$.

Next, we prove that $x \notin A \implies f(x) \notin B$:

If G does not have a clique of size k, then any subset $S \subseteq V$ of size k does not form a clique. Then, there exists at least one pair $a, b \in S$ such that $M[a, b] = 0$. Therefore, the total deliciousness of S is less than $\dfrac{k(k-1)}{2} = m$.

Since $x \in A$ if and only if $f(x) \in B$, and the reduction f runs in polynomial time, the Best-Dish problem is NP-hard.

We have shown that the Best-Dish problem is in NP and NP-hard, so it is NP-Complete.

Problem 4
(25 points)

Show that the following `FILL-BINS` problem is NP-complete.

Input: You are given m bins with weight capacities c_1 to c_m. You are given n items with weights w_1 to w_n.
Output: Whether or not it is possible to put every item in a bin without exceeding the weight capacity of any bin.

Solution: To prove that the FILL-BINS problem is NP-Complete, we follow the following logic:

We recognize we can verify whether a given assignment of items to bins does not exceed the capacity of any bin in poly-time.

We begin by checking that every item is assigned to some bin. Then, for each bin i, we compute the total weight of items assigned to it by summing the weights w_j of the items assigned to bin i. We verify that this total weight does not exceed the bin's capacity c_i.

We observe that there are n items and m bins. For each item, assigning it to a bin takes $O(1)$ time. For each bin, summing the weights of its assigned items takes $O(n)$ time (since in the worst case, all items could be assigned to one bin). Therefore, the total time complexity is $O(n + m \cdot n) = O(mn)$, which is polynomial. And so, the problem is in NP.

We reduce the Subset Sum problem, which is known to be NP-Complete, to the FILL-BINS problem.

Given an instance $(\{w_1, w_2, \ldots, w_n\}, T)$ of the Subset Sum problem, we construct an instance of the FILL-BINS problem as follows:

We first create two bins, so $m = 2$.Then, we set the capacities of the bins to:

$$c_1 = T, \quad c_2 = \sum_{i=1}^{n} w_i - T.$$

Then, we use the same set of items with weights $w_1, w_2, \ldots, w_n$.

We find computing the total sum $W = \sum_{i=1}^{n} w_i$ takes $O(n)$ time. Similarly, setting up the bins and copying the item weights takes $O(n)$ time. Therefore, the reduction runs in poly-time.

Now, we prove that $x \in A \implies f(x) \in B$:

If there exists a subset $S \subseteq \{w_1, w_2, \ldots, w_n\}$ such that $\sum_{w_i \in S} w_i = T$, then we assign the items in S to bin 1 and assign the remaining items $\{w_1, w_2, \ldots, w_n\} \setminus S$ to bin 2. We find

the total weight in bin 1 is $\sum_{w_i \in S} w_i = T = c_1$ and the total weight in bin 2 is:

$$\sum_{w_i \notin S} w_i = \sum_{i=1}^{n} w_i - \sum_{w_i \in S} w_i = W - T = c_2.$$

Therefore, we find neither bin exceeds its capacity, and all items are assigned to bins. Thus, $f(x)$ is a "yes" instance of the FILL-BINS problem.

Now, we prove that $x \notin A \implies f(x) \notin B$:

If there does not exist a subset $S \subseteq \{w_1, w_2, \ldots, w_n\}$ such that $\sum_{w_i \in S} w_i = T$, then any assignment of items to bins must assign the items to bins 1 and 2. Let us suppose we attempt to assign items to bins without exceeding capacities. Since no subset sums to T, it is impossible to fill bin 1 exactly to capacity without exceeding it. Therefore, assigning any subset of items to bin 1 will either: sum to less than T, leaving unused capacity in bin 1 and exceeding capacity in bin 2 (since $c_1 + c_2 = W$), OR sum to more than T, exceeding bin 1's capacity.

Therefore, it is impossible to assign all items to bins without exceeding at least one bin's capacity. Therefore, $f(x)$ is a "no" instance of the FILL-BINS problem.

Since $x \in A$ if and only if $f(x) \in B$, and the reduction f is computable in polynomial time, the FILL-BINS problem is NP-hard.

Since we have shown that the FILL-BINS problem is in NP and since we reduced the NP-Complete Subset Sum problem to it, we find the FILL-BINS problem is NP-Complete.